

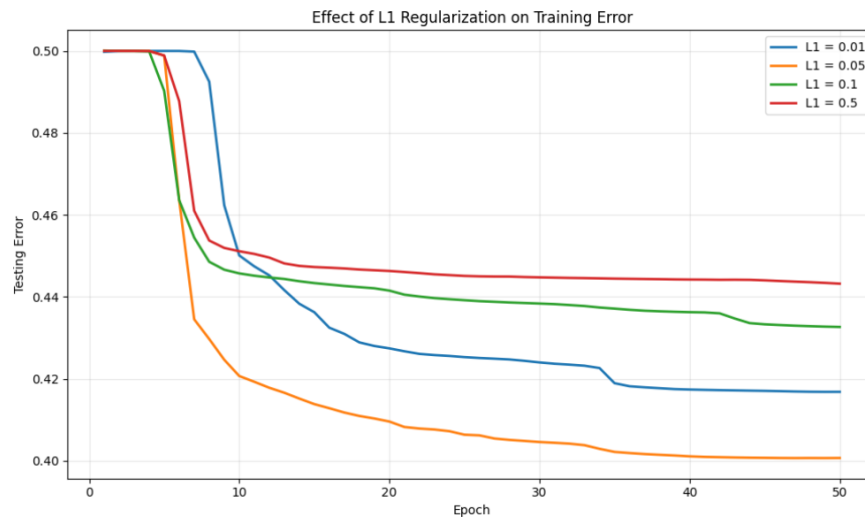
- Smooth gradient don't work,It is proven
- L1Loss don't work.It is proven
- gradient decay don't happen much
- When I see the distribution of my training I see the a lot of weights is saturated in the left side,Not a lot of weights saturated in the right side,It is because of the regulation, So I am thinking Do I need to regulate from both side, I feel like it. Because we have seen having w.relu() error yield better result
- I removed DecreaseLRonPlateau in scheduler,SO I think i need to bring it back,I am not sure how improve it make it.
- Increased cost in regularization always make the model better in avg.
- Always use lower learning rate,Default ADAM settings is giving better result than scaled one
- Lower reg is always bad on avg,Not sure why
- Will try increasing as much as possible to see the effect.

All of this test is happening 1 time,So it is not confident the result are authentic,Maybe some random make the test better.

Newly Initialized model.

Experiment 1:

1. Currently I have initialization for having mean =1 for even layer,and mean =0 for odd layer,I am now testing how much regularization helps in here.
2. had MSE error
3. I had a batch size of 256,Higher than 128
4. use adam optimizer,with default learning rate,Because I think low learning rate works best.
5. Their is a balance on what regularairzation works better.Here 0.05 works the best.

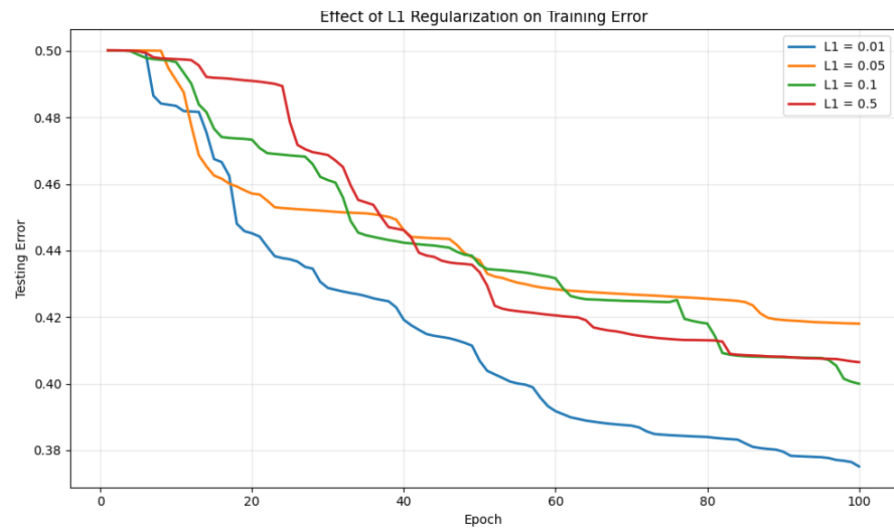


6. So I think using differnet loss function will influence the result differently

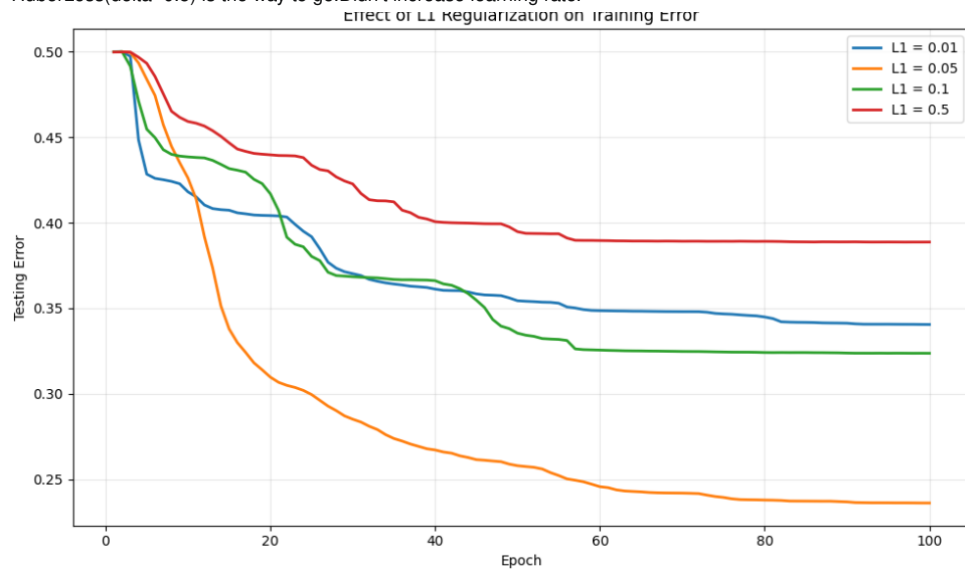
Newly initialized model

Experiment 2:

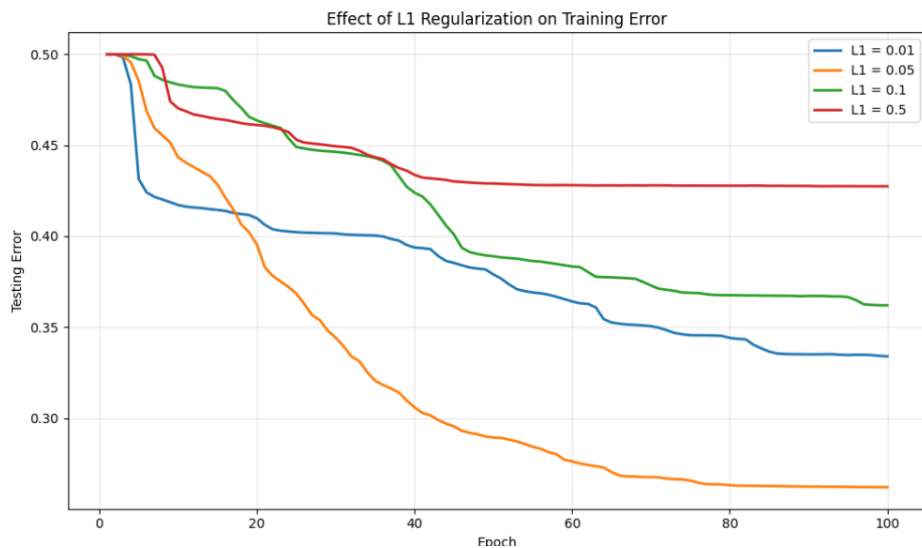
1. I aam gonna do the entire experiment on experiment 1,But with HuberLoss,I think this will work better.
 2. batch size 128,Making some more rng.
 3. Test which regularization works best.
 4. Doing 100 epoch of training.
 5. In simple glance,the HuberLoss is doing ridiculously good,compared to MSELoss,It can because of both have different initialization.Or because it have more randomization because of 128 batch size.
- HuberLoss(delta=0.1) results



HuberLoss(delta=0.5) is the way to go. Didn't increase learning rate.

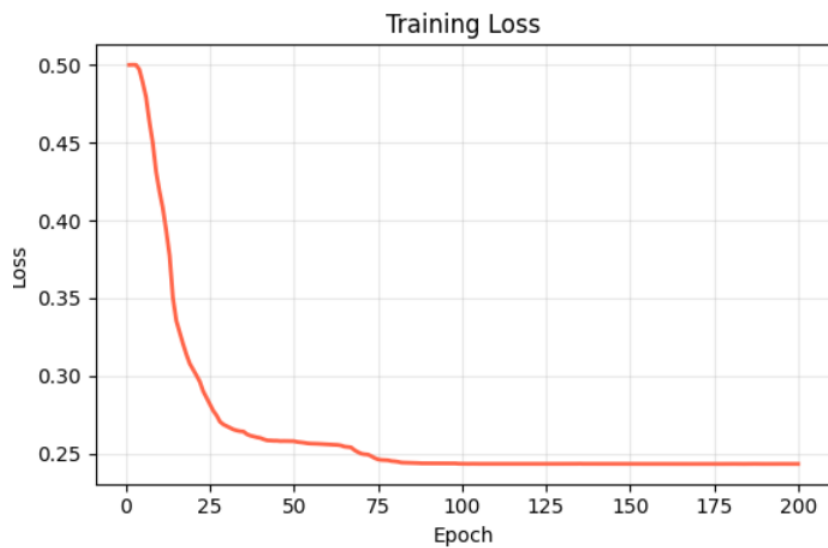


I increased the dimension to see how the performance get better. Previously I had (256, 128, 64, 32) and now I have (256, 128, 64, 128, 64, 32),



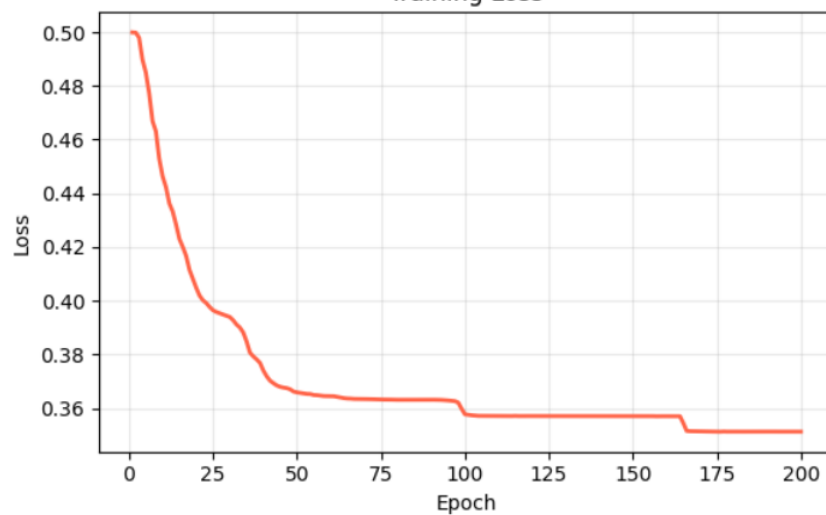
Experiment 3:

With different test it seems like low learning rate is doing best for HuberLoss.



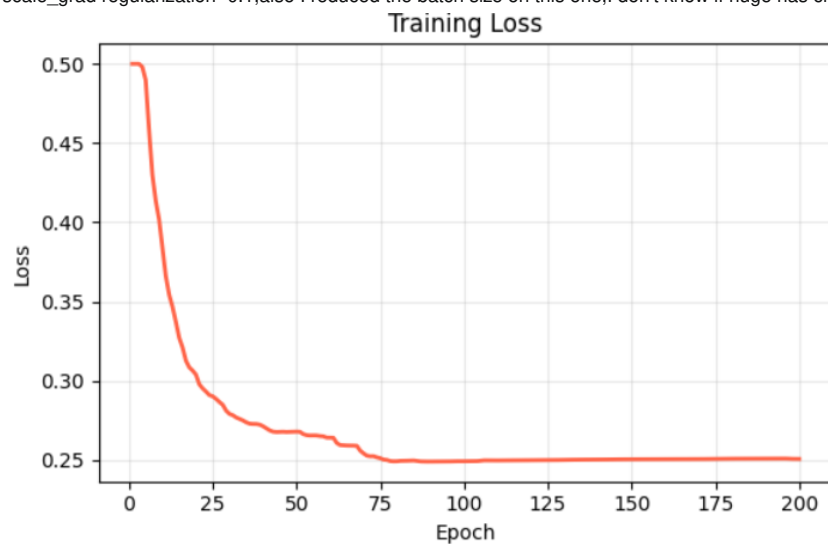
Experiment 4:

I increased the layer count (256, 128, 64, 128, 64, 32) seeing the [feedback](#). As before, it plateaued faster.



Experiment 5:

I changed the error to MSE and with increased layer mSE worked somewhat better, than HuberLoss(0.5), with `scale_grad regularization=0.1`, also I reduced the batch size on this one, I don't know if huge has changed.



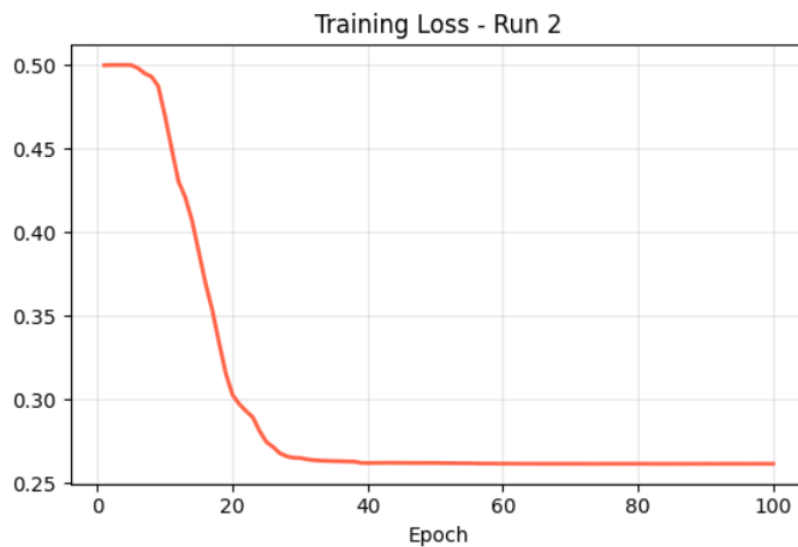
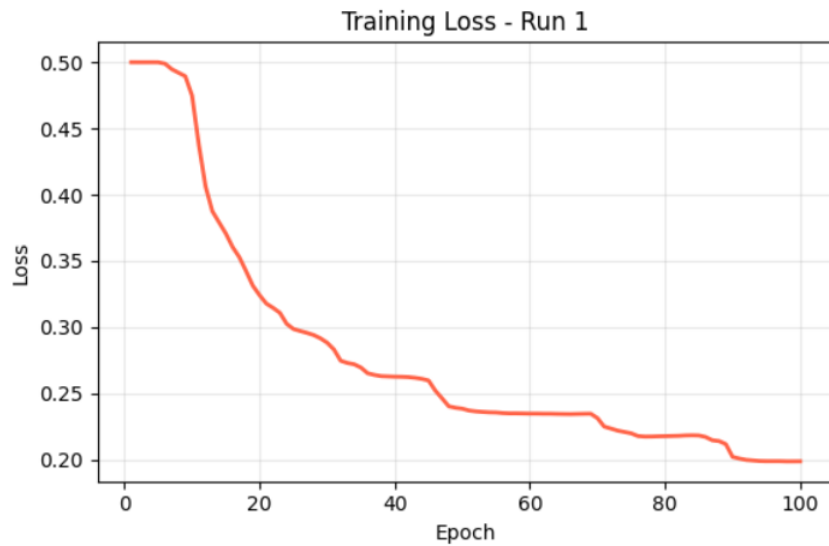
Still it wasn't better, Let's remove the `scale_grad` and see the result.

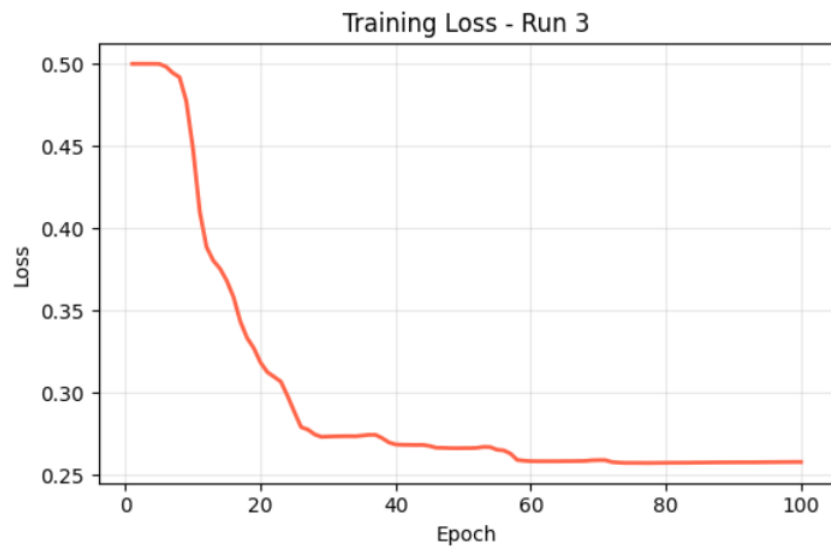
Experiment 6:

Ok having

1. grad_scalar=True,
2. higher layer (256, 128, 64,128,64 ,32),
3. and higher regularization (0.5,0.5,0.5)
4. MSE error,
- 5.

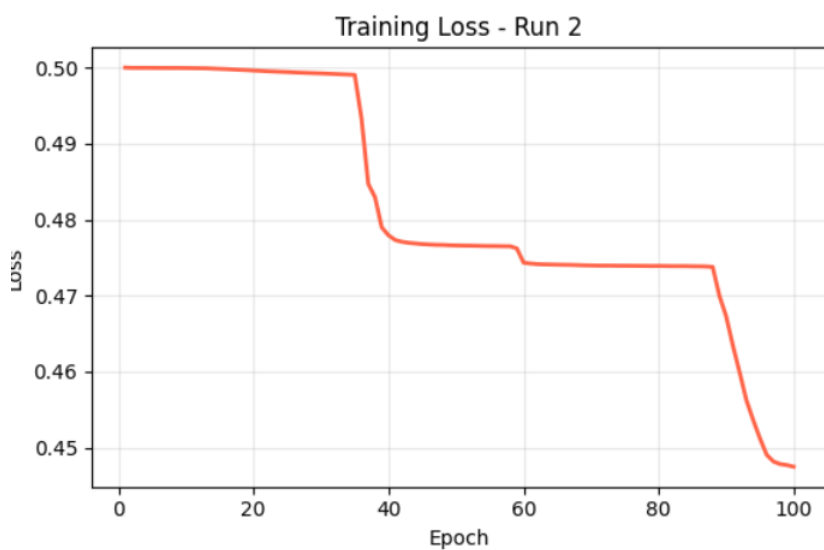
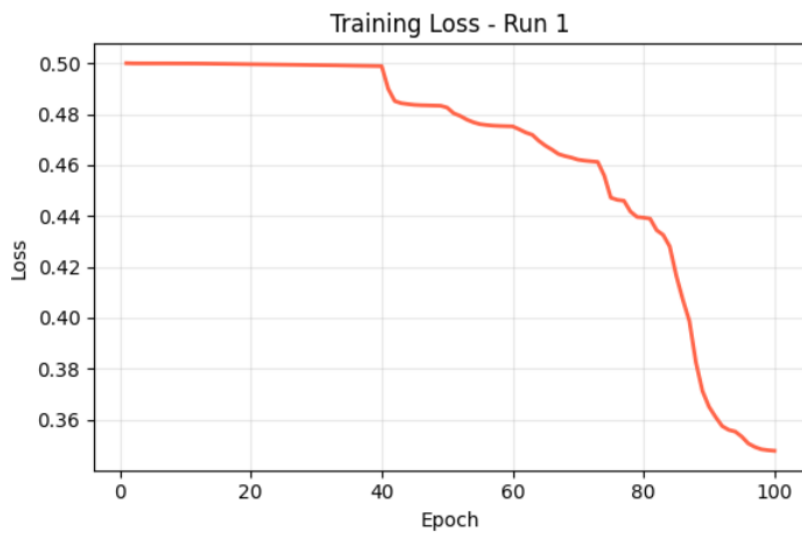
I have made the model learn much better. Given I have kept the default optimizer settings, result:As it turns out the this approach has a lot of plataued values. Some time the result is not good as expected.

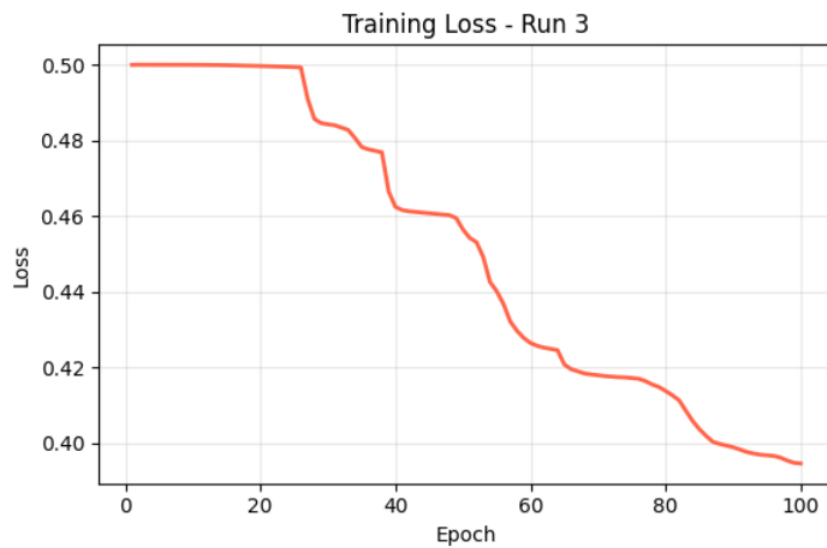




Experiment 7:

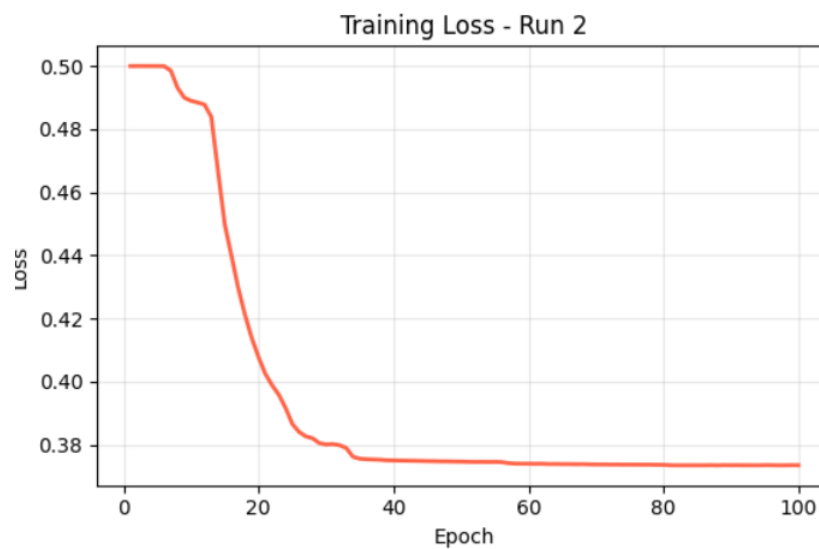
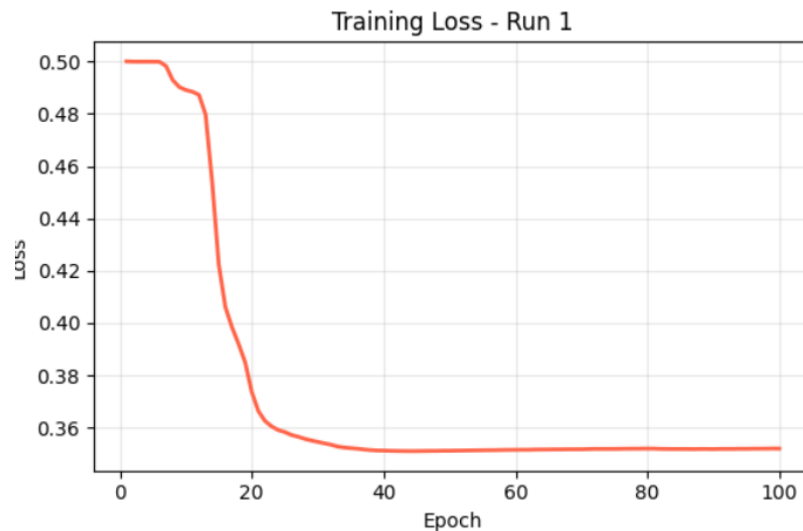
Testing if changing the grad_scalar has positive or negative [effect](#). It turns out current grad_scalar has good effect on the optimization given the regularization is high enough, Current test similar to experiment 6, but with no grad scalar, Making.





Experiment 8:

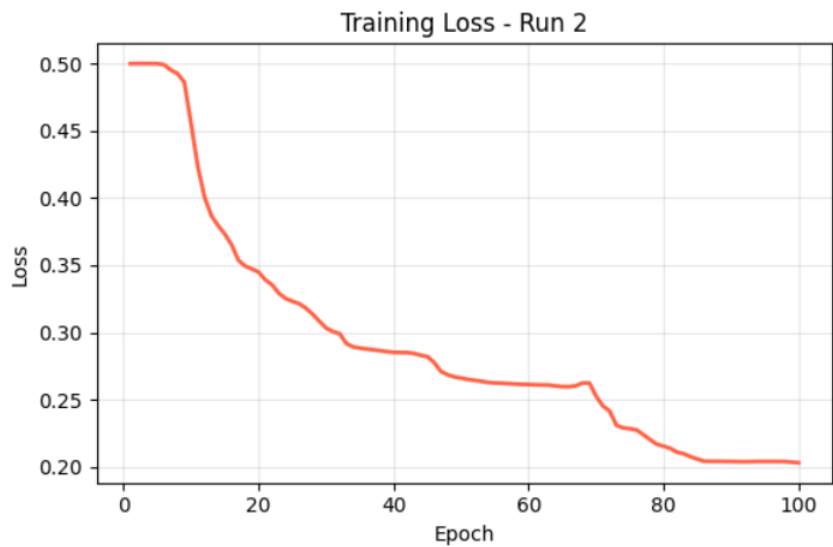
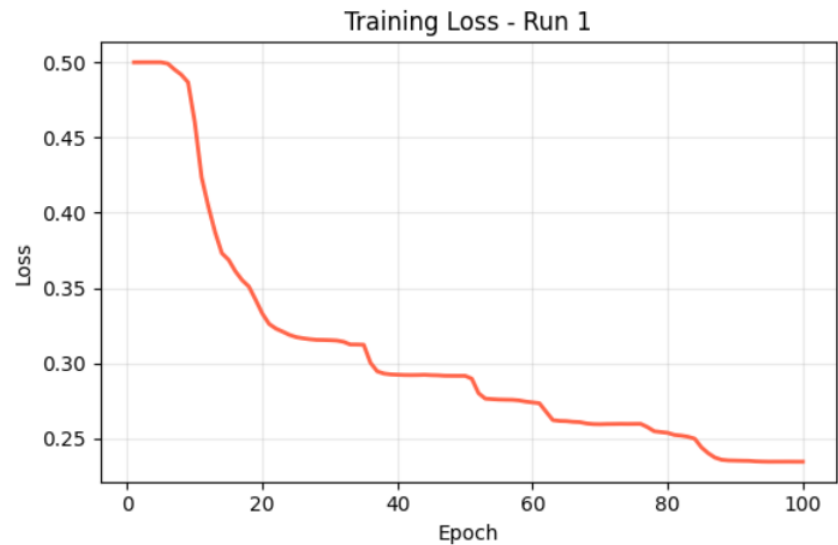
Because I previously find out `grad_scalar` really works great, I am gonna keep it in for the rest of the project, Current test was doing if is compared to regularization 2 if regularization 1 is decreased, So say (0.4, 0.5), Will the training go bad, My previous research said it will, Now I am checking anew. Got better result.



It seems that regularization1 can't never be less than regularization2, I need to test what happens on same or higher.

Experiment 9:

I kept the $\text{reg1}=0.4$ and decreased $\text{reg2}=0.4$ so like to prove their is no effect of $\text{reg1}=0.5$, I am getting the good behaviour like before,Now I need to check if $\text{reg1}>\text{reg2}$ Willl their be more effect or not.

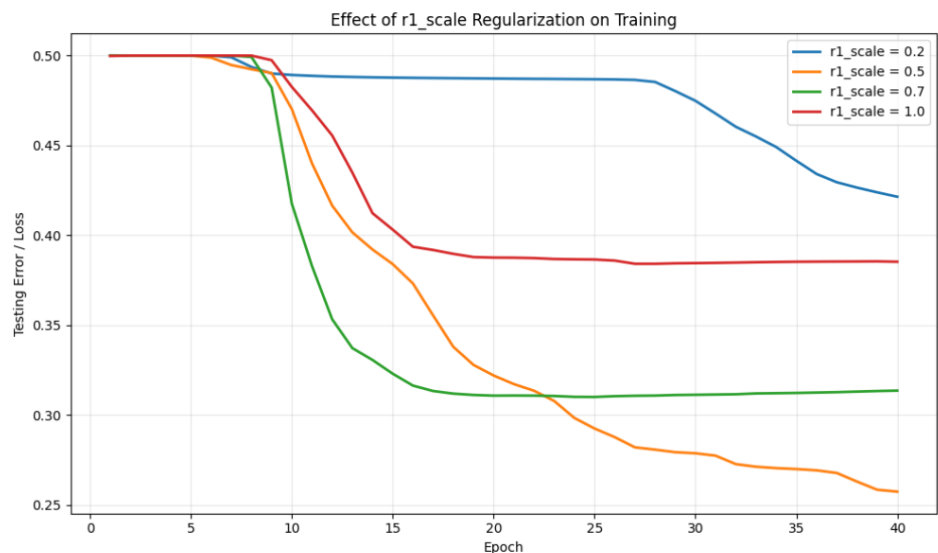


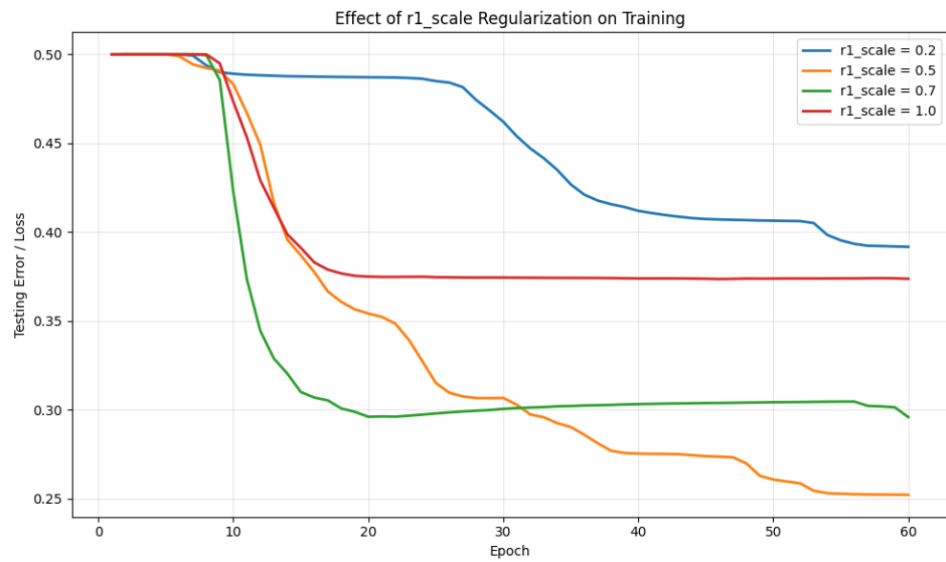
came to the position it was before.,

It

Experiment 10:

Wanna see if I increase reg_1 , Will it get as much performance. Remember I am keeping regularization 2 constant, This test check what is the behaviour of reg_1 if I keep reg_2 constant, Here $\text{reg2}=0.5$,

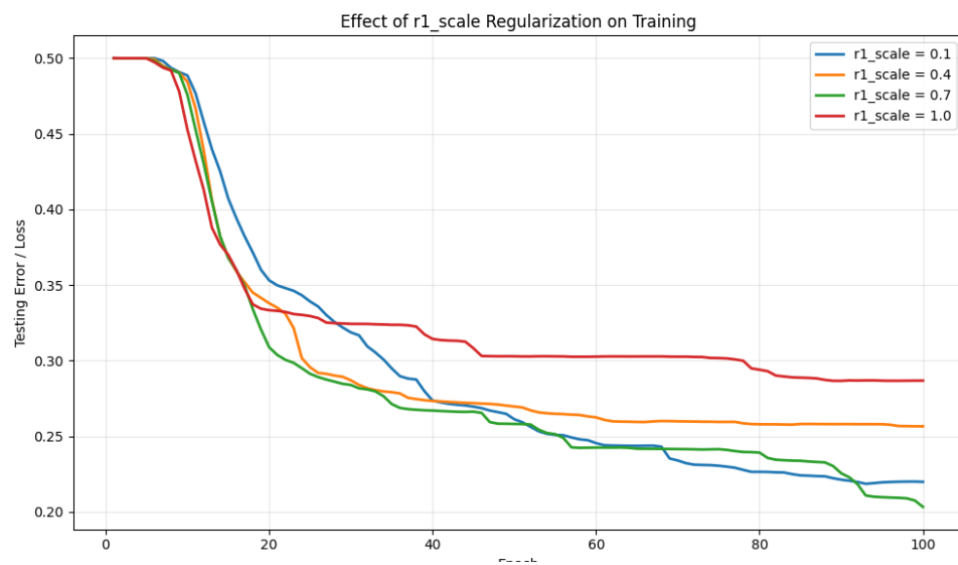
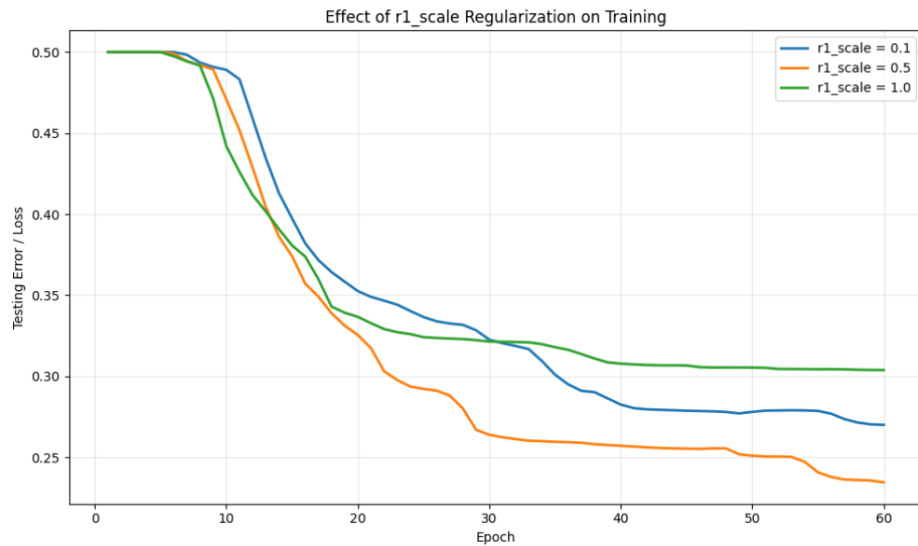


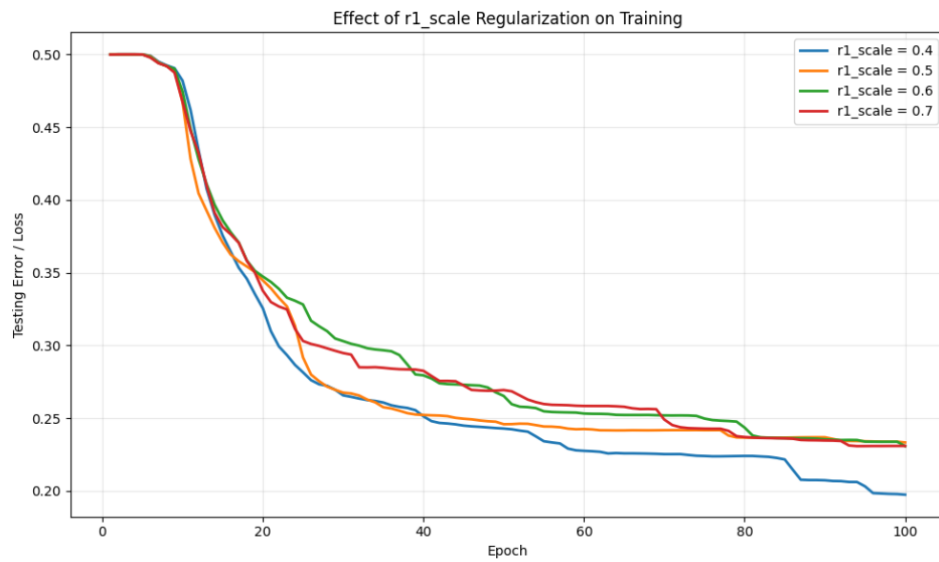


The test always gives that having $r1=r2$ regularization is the way to go.

Experiment 11:

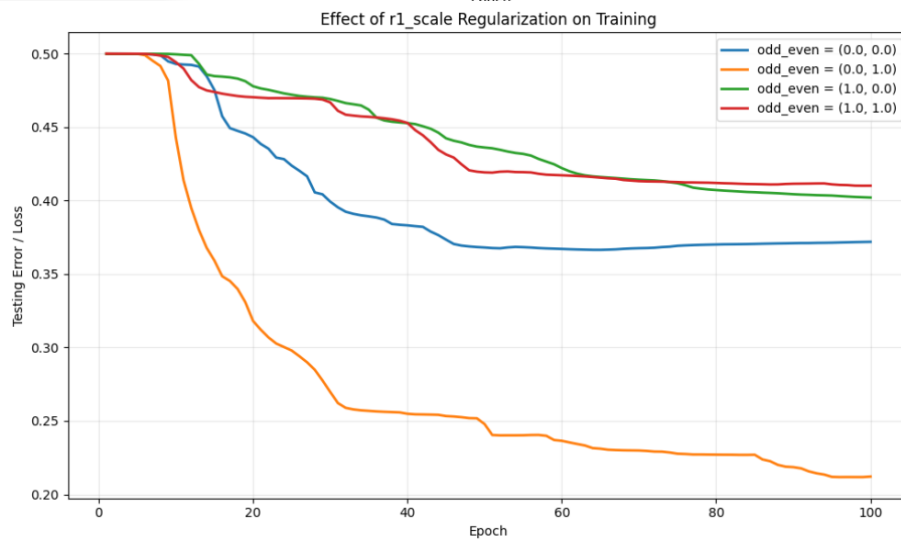
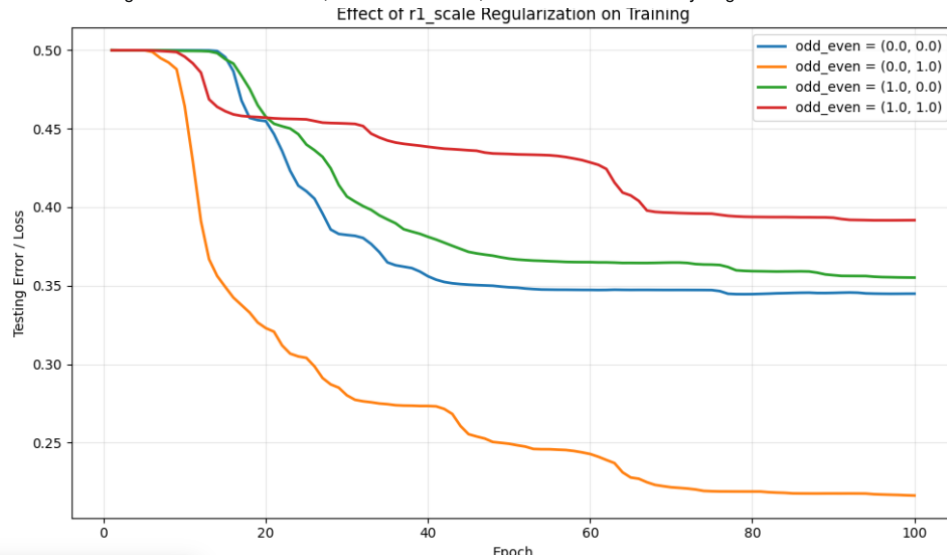
Checked which regularization works the best, as it turns out $reg=0.5$ works the best, like always.





Experiment 12:

Checked the right initialization method, It turns out odd=0, even=1 biased is the way to good



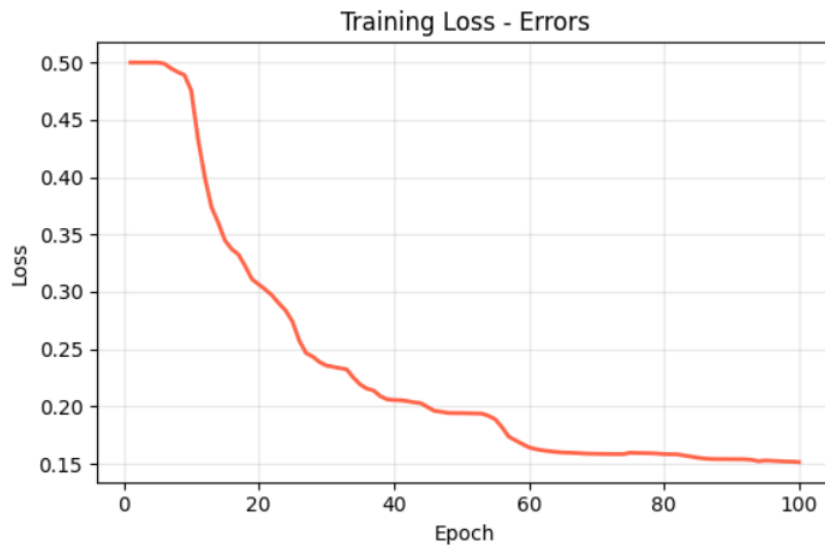
Now other than normal distribution, I changed it to constant, And I got really bad performance in result. Plot isn't working, Even with 100 epoch I got .4999 So like random choice.

But there might be a sweet spot, Like If I have odd_even=(0.5,0.5) normal distribution,

Epoch 0099 | loss = 0.469072 | error = 0.347975 | reg = 0.302760 | tau_0 = 13.096493 | tau_1 = 19.476036 | tau_3 = 9.336477 | tau_4 = 8.504196 | tau_5 = 7.956133

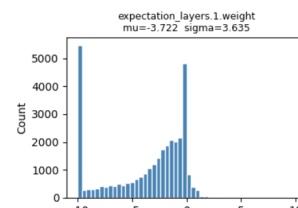
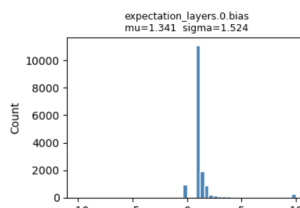
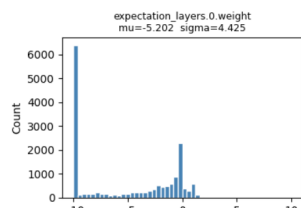
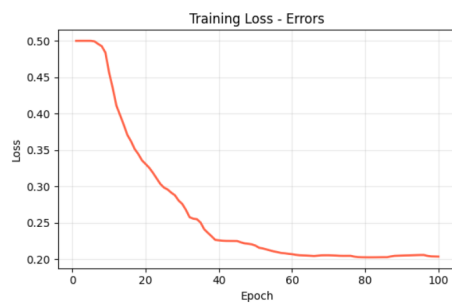
Experiment 13:

Changed the regularization,where it toggles when the training platuaed,Got good result



Don't

get good result always,

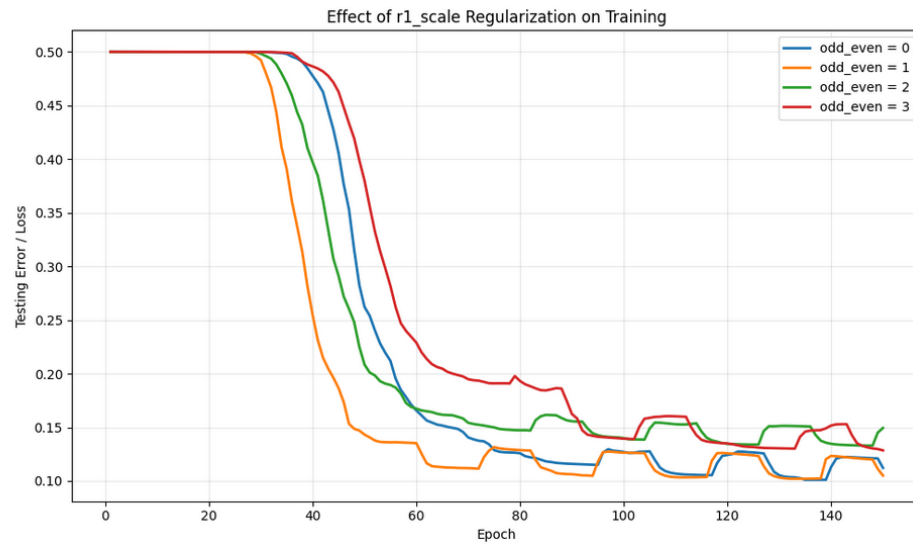


I guess it has high rng dependent,Like

Epoch 0229 | loss = 0.104137 | error = 0.232825 | reg = 0.000000 | tau_0 = 6.900964 | tau_1 = 11.14.299672 | tau_3 = 8.728804 | tau_4 = 10.289113 | tau_5 = 7.882086

In long run you can see, the error just isolate in the plateau.

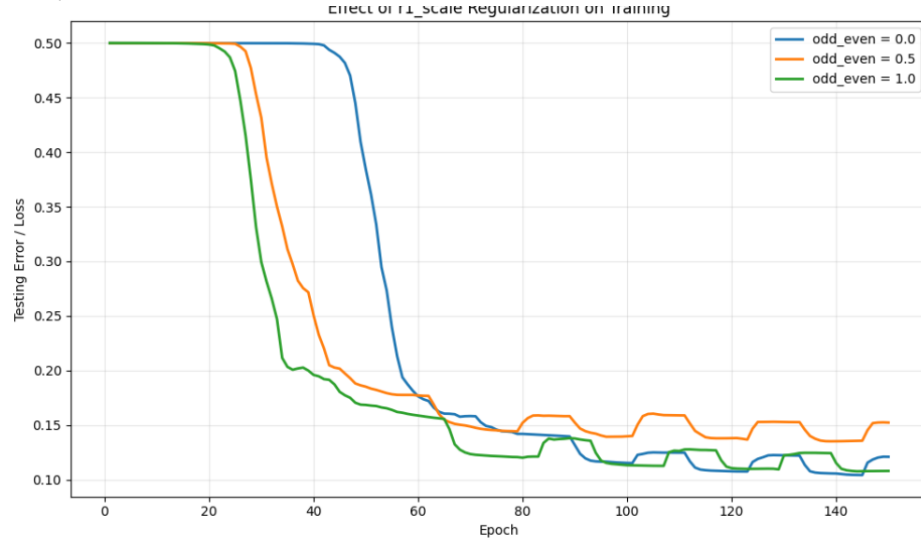
===== COMPLETED RUN WITH odd_even=3 =====



,So maybe it doesn't have as much effect we think it has.

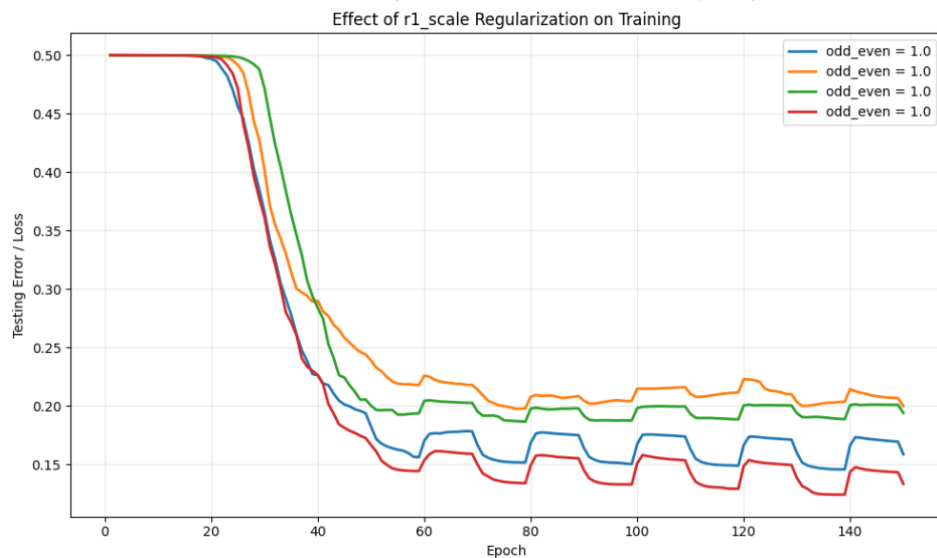
Experiment 14:

did the experiment with various bias, To see the effect, It turns out bias don't have much effect



Experiment 15:

Iterative randomization, where I switch from 0 regularization to 1 and vice versa every 10 epoch,



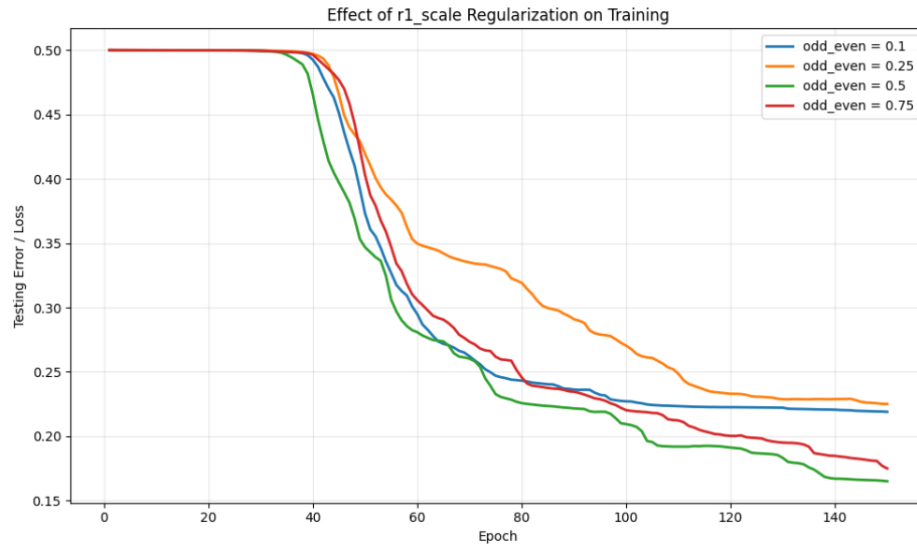
Didn't work well.

Experiment 16:

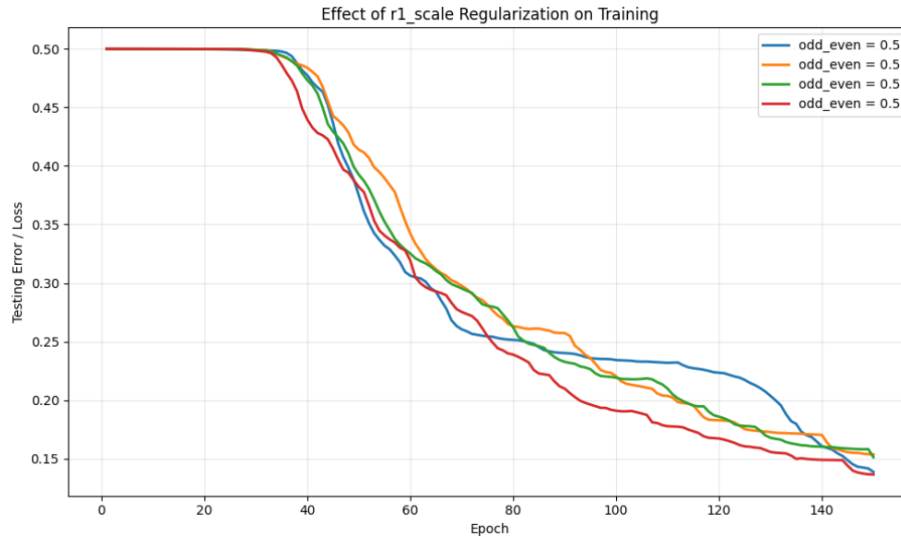
I have another assumption, That was If I have missed something in the initialization. So I made the initialization (0.2,0.8),and turns out it doesn't have any considerable effect.

Experiment 17:

I made a new regularization, In the previous regularization I have gradient when the weight is greater than 1, To get close to 1, In this new regularization I removed that, new on $w > 1, \text{grad} = 0$, Also I removed isolation (regularization turns on and off on plateau) in the new experiment.

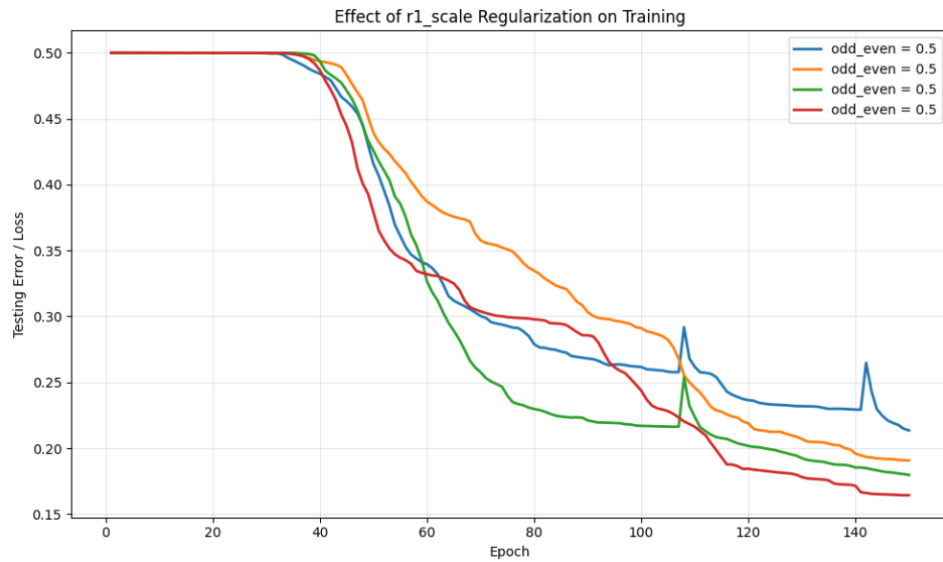


Then I tested one with isolation on, It seems like isolation has no useful effect, But in all this time this new regularization is working good.



Experiment 18:

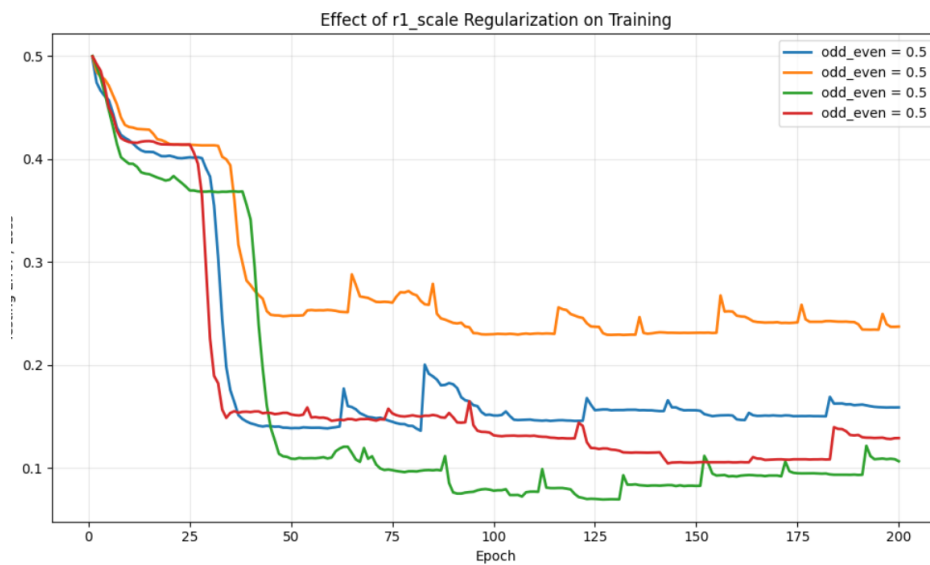
I have seeing that thins new regularization is working the best, On second thought, I am using random weight on platua, It is working somewhat.



Let's make the training bigger to see what's the issue. If I train for 300 epoch, What will I get, Also I think I need to update the learning rate, In the graph you see the node can keep going, I am gonna test with 300 epoch to see what happens. Also we can see the randomization of weights on platua do help sometimes, But I also found out Unlike previous regularization, This one have a lot of weights between 0.5 and 1.

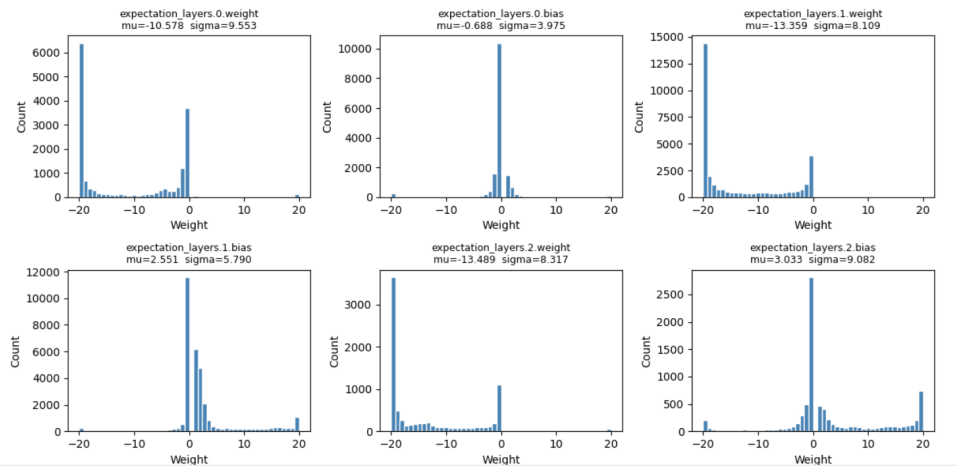
Experiment 19:

In the new test I am checking if I move th regularization between $\sqrt{\cdot}$ and $\cdot$ it starts with $\cdot$ To make sure the weights remains in 0 and 1,



As you can see it crosses the 0.1 boundary, also kinda fast, But a issue is it is also highly skewed toward initial randomaization.

Other than isolation of regularization, there is also randomization on platua, So a lot of things. The weight distribution in here, is also spot on



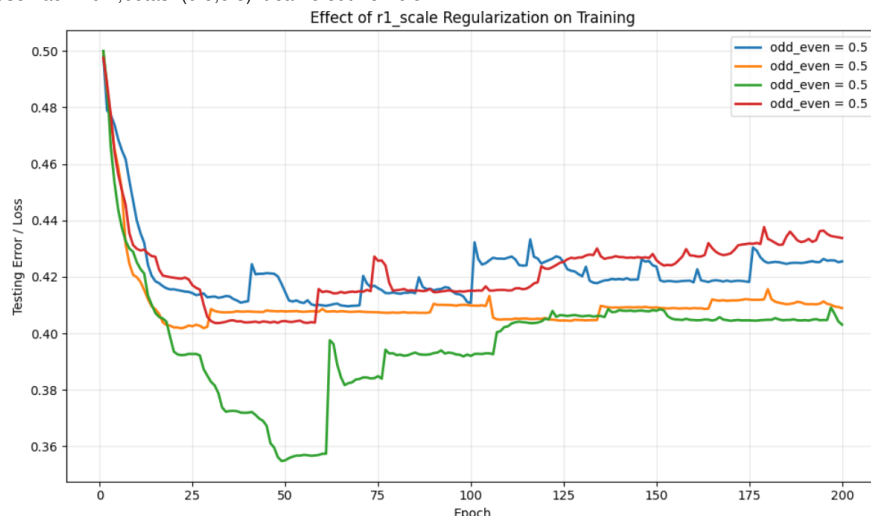
But we also see not all initialization make through.

I forgot I also made the lr:0.1 and betas:(0.5,0.5) here.

Experiment 20:

only keep $_/_$ regularization and raddom noise added.

- also had lr=0.1,betas=(0.5,0.5) It came out horrible



Experiment 21:

I am gonna do experiment 19 but with default learning rate and default betas I guess adding noise is halting the progress,Didn't go well.

Experiment 22:

I am now experimenting more with and gate,and which type result in better gradient distribution, Here is the image

